

Coding for Artificial Intelligence

Assignment 6

AI Social Media Auditor

Name: Tanishka Hira

Assignment: social_media_auditor.py

Date: 07-09-2026

Overview

The purpose of this assignment is to develop a small Python program that demonstrates the use of dictionaries and sets for handling textual information and user data.

The program is divided into two main sections. The first section examines a paragraph and records how frequently each word appears. The second section compares follower information from two different social media platforms.

Several test cases are also included to verify that the program behaves correctly when unusual or boundary inputs are provided.

Task 1: Word Frequency Analysis

The first section processes a paragraph and creates a dictionary containing each word along with the number of times it occurs.

Before a word is stored, it is converted into lowercase and its surrounding punctuation is removed. This prevents words such as **AI**, **ai**, and **AI**, from being treated as three separate entries.

The dictionary is updated using `setdefault()`. If a word has not appeared previously, its initial value is set to zero and then increased by one.

Task 2: Follower Comparison

The second section works with two lists containing follower IDs. Both lists are converted into Python sets so that set operations can be applied to compare the users.

Three different relationships between the two collections are examined:

- **Common users:** The intersection operation identifies followers present in both collections.
- **Platform-specific users:** The difference operation determines which users occur in one platform but not the other.
- **Single-platform users:** The symmetric difference identifies users who occur in exactly one of the two sets.

The direction of subtraction is important because `set_a - set_b` and `set_b - set_a` can produce different results.

Source Code

The following is the complete implementation of the `social_media_auditor.py` program.

```
1 """
2 Program: AI Social Media Auditor
3 Purpose: Perform word-frequency analysis on text and compare follower
4           lists from two social media platforms using dictionaries and sets.
5
6 Author: Tanishka Hira
```

Date: 07-09-2026

Task 1: Word Count

Before counting words, we need to clean up the text. Python treats words differently based on capitalization and punctuation so "AI", "ai", and "AI," would end up as separate entries. To fix this, we convert everything to lowercase and strip off surrounding punctuation before logging each word's frequency.

Task 2:

When comparing follower lists across two platforms, Python sets make it straightforward to spot overlapping and unique users:

- i. Users on both platforms: Found using set intersection (set_a & set_b).
- ii. Platform A only: Found using set difference (set_a - set_b). Keep in mind that order matters here set_a - set_b yields users unique to Platform A, whereas set_b - set_a isolates users on Platform B.
- iii. Platform B only: Found using set difference (set_b - set_a).
- iv. Single-platform users: Found using symmetric difference (set_a ^ set_b). This extracts everyone who follows on only one platform, without breaking down which specific platform they use.

"""

'''--- Task 1 ---'''

```
import string
```

```
text = ("AI is changing the world. ai is powering new tools, and AI is helping  
researchers learn faster. The world loves AI, and the world depends on it more  
each year.")
```

```
word_counts = {}
```

```
for raw_word in text.split():  
    cleaned_word = raw_word.lower().strip(string.punctuation)  
  
    if not cleaned_word:  
        continue  
  
    word_counts.setdefault(cleaned_word, 0)  
    word_counts[cleaned_word] += 1
```

```
print("--- TASK 1: WORD FREQUENCY COUNTER ---")
```

```
for word, count in word_counts.items():  
    print(word, ":", count)
```

'''--- Task 2 ---'''

```

49
50 platform_a_followers = ["u101", "u102", "u103", "u104", "u105"]
51 platform_b_followers = ["u103", "u104", "u106", "u107"]
52
53 set_a = set(platform_a_followers)
54 set_b = set(platform_b_followers)
55
56 both_platforms = set_a & set_b # intersection
57 only_a = set_a - set_b         # unique to Platform A
58 only_b = set_b - set_a         # unique to Platform B
59
60 either_only = set_a ^ set_b    # only_a | only_b (symmetric difference
    operator)
61
62
63 print("\n")
64 print("-- TASK 2: The Follower Deduplicator --")
65 print("Follows on both platforms (&): ", both_platforms)
66 print("Unique to Platform A (-): ", only_a)
67 print("Unique to Platform B (-): ", only_b)
68 print("Unique to exactly one platform (^): ", either_only)
69
70
71
72 '''-- Edge Cases to Handle --'''
73 '''1. An empty paragraph'''
74
75 print("Edge Cases to Handle")
76 text = ""
77 word_counts = {}
78
79 for word in text.split():
80     word = word.lower().strip(string.punctuation)
81
82     if not word:
83         continue
84
85     word_counts.setdefault(word, 0)
86     word_counts[word] += 1
87
88 print("1. An empty paragraph: ", word_counts)
89
90 '''2. A word that is punctuation only'''
91 text = "--"
92 word_counts = {}
93
94 for word in text.split():
95     word = word.lower().strip(string.punctuation)
96
97     if not word:
98         continue

```

```

99
100     word_counts.setdefault(word, 0)
101     word_counts[word] += 1
102 print("2. A word that is punctuation only: ", word_counts)
103
104
105 '''3. Two follower lists with no overlap at all'''
106 platform_a_followers = ["u101", "u102", "u103"]
107 platform_b_followers = ["u104", "u105", "u106"]
108
109 set_a = set(platform_a_followers)
110 set_b = set(platform_b_followers)
111
112 both_platforms = set_a & set_b
113 only_a = set_a - set_b
114 only_b = set_b - set_a
115
116 print("3. Two follower lists with no overlap at all;")
117 print("Both:", both_platforms)
118 print("Only A:", only_a)
119 print("Only B:", only_b)
120
121
122 '''4. Two identical follower lists'''
123 platform_a_followers = ["u101", "u102", "u103"]
124 platform_b_followers = ["u101", "u102", "u103"]
125
126 set_a = set(platform_a_followers)
127 set_b = set(platform_b_followers)
128
129 both_platforms = set_a & set_b
130 only_a = set_a - set_b
131 only_b = set_b - set_a
132 print("4. Two identical follower lists; ")
133 print("Both:", both_platforms)
134 print("Only A:", only_a)
135 print("Only B:", only_b)

```

Results

The word-count section successfully creates a dictionary containing the normalised words and their respective frequencies.

For the supplied follower data, the common users are:

$$\{u103, u104\}$$

The users found only in Platform A are:

$$\{u101, u102, u105\}$$

The users found only in Platform B are:

$$\{106, u107\}$$

Therefore, the users appearing on exactly one platform are:

$$\{u101, u102, u105, 106, u107\}$$

The program uses the value 106 exactly as written in the provided Platform B list.

Edge Case Verification

Additional inputs were included to check the reliability of the program.

1. **Empty input:** When the paragraph contains no text, the resulting dictionary remains empty.
2. **Only punctuation:** A token containing only punctuation is removed during cleaning and is not inserted into the dictionary.
3. **No common followers:** When the two follower collections contain completely different IDs, their intersection is empty and all users are classified as platform-specific.
4. **Matching follower collections:** When both lists contain the same users, every user belongs to the intersection and neither platform has a unique user.

Submission Note

The program was executed with the given text and follower data to check the behaviour of both tasks. The text-processing portion successfully converted the words to lowercase and removed punctuation before updating their frequencies. This allowed differently formatted versions of the same word to be grouped together. For the follower comparison, the two lists were converted into sets and the required set operations produced the expected groups of common and platform-specific users. The symmetric difference was used to combine users who appeared on only one platform. I also tested the program with an empty paragraph and a punctuation-only token to ensure that these inputs did not create unwanted dictionary entries. Finally, separate follower lists with no common users and identical lists were tested. The results were consistent with the expected behaviour of each operation.